UMnatural — Guía del proyecto (estado al 9 de noviembre de 2026)

Equipo **SoftSolum** · Proyecto TIC 2026 · Universidad de Montevideo Todo lo que hay que saber para trabajar, demostrar y defender el sistema. En especial, **lo nuevo desde el 17 de octubre**.

---

1. Resumen en 30 segundos

- **Qué es:** una plataforma donde los **prestadores** (empresas o particulares) publican actividades turísticas, un **administrador** los verifica y los **turistas** buscan y reservan.
- **Tecnología:** Java 21+ · Spring Boot 4.1.1 · Maven · JPA/Hibernate · Thymeleaf · PostgreSQL en la nube (**Neon**) · Lombok · BCrypt · JUnit 5 + Mockito.
- **Estado del código:** todo está en `main`. El último PR es el **#22, "Implementación P2"**, del 9 de noviembre.
- **Ya funciona de punta a punta:** registro de prestador → el admin acepta → el prestador crea una actividad y sus horarios → el turista se registra, busca y reserva → el prestador ve la reserva.
- **Falta:** el pago (pasarela externa), comentarios y valoraciones, la API pública con su documentación, el clima (opcional) y parte de la documentación.

---

2. Qué pasó desde el 17 de octubre

Fecha	Qué se hizo
18–19 oct	Formularios de registro de prestador (Ori), login (Juan Diego), validaciones de RUT, CI y teléfono (Ori), verificación de contraseña repetida (Tefi), varios merges y arreglos de conflictos (Joaquín)
19 oct	**Migración de H2 a PostgreSQL en Neon** (Juan Diego + Joaquín), PR #19–#21
19 oct – 8 nov	Trabajo local sin subir: datos completos del emprendimiento en el panel admin, primera versión del turista
8 nov	Relevamiento de lo que falta y reparto del código en 4 bloques (`reparto-4-personas.md`)
9 nov	**Bloque P2 (turista):** explorar, detalle, reservar, mis reservas, vencimiento a las 2 h
9 nov	**Actividades del prestador** (alta, edición, baja, horarios, reservas recibidas) y **tipos del admin**
9 nov	Arreglos: login sin distinguir mayúsculas, el emprendimiento guarda email, teléfono y descripción, links para volver al inicio, tipos y equipamiento escritos a mano

Fecha	Qué se hizo
9 nov	**Commit “Implementación P2” + PR #22 mergeado a `main`**

---

3. Cómo correr el proyecto

Requisitos

- **IntelliJ IDEA**. Ojo: **no PyCharm**, que no tiene Java y además rompe la configuración del proyecto (ver 3.3).
- JDK 21 o superior (se usa Temurin 26).
- La contraseña de Neon, que se pasa por privado y **nunca va en el repo**.

Pasos

1. **File → Open →** elegí el archivo `proytic2026/pom.xml` → **Open as Project**.
2. Esperá a que Maven termine de cargar.
3. Abrí `Proytic2026Application.java` → ▶ al lado del `main`.
4. **Edit Configurations → Environment variables:** `DB_PASSWORD=<contraseña>`.
5. ▶ **Run**. Cuando aparezca `Started Proytic2026Application`, entrá a **http://localhost:8080**.

Problemas típicos

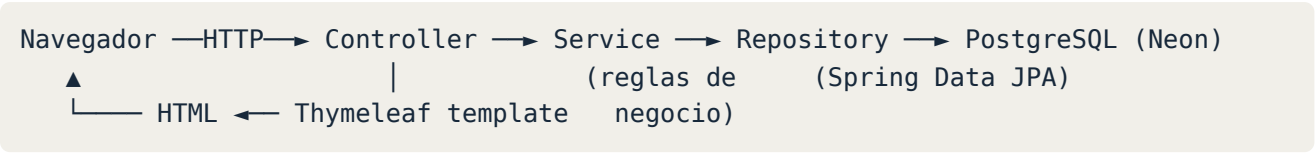
Síntoma	Causa	Solución
`password authentication failed for user 'neondb_owner'`	Falta `DB_PASSWORD` o es incorrecta	Ponerla en Edit Configurations; si la resetearon en Neon, pedir la nueva
`SocketTimeoutException` / no conecta	La red bloquea el puerto 5432 (pasa en la facultad)	Usar el **hotspot del celular**. Llevarlo el día de la defensa
La configuración aparece como “Unknown” / no existe la opción “Application”	El proyecto se abrió con PyCharm	Cerrar, borrar o renombrar la carpeta `.idea` y abrir el `pom.xml` con IntelliJ
`package org.junit... does not exist` y cientos de errores	IntelliJ no reconoce el proyecto como Maven	Clic derecho en `pom.xml` → Maven → **Sync Project**; si no alcanza, borrar `.idea` y reabrir
`null value in column "xxx" violates not-null constraint`	Columna vieja en Neon que el código ya no usa	En el SQL Editor de Neon: `ALTER TABLE <tabla> DROP COLUMN IF EXISTS xxx;`

Datos de prueba

- **Admin:** `admin@umnatural.uy` / `admin123`. Lo crea `AdminInicializador` al arrancar.
- **Cédulas válidas:** `12345672`, `45678905`, `11111111`, `22222222`, `33333333`.
- **RUT válido:** `212345670015`.
- **Celular:** formato `09XXXXXXX`, por ejemplo `099123456`.

---

4. Arquitectura (MVC en capas)

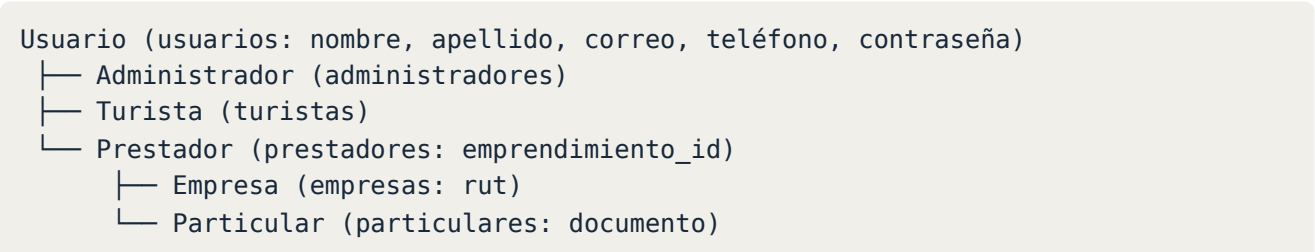


Capa	Paquete	Responsabilidad
Model	`entities`	Clases JPA = tablas (`@Entity`)
Repository	`repository`	Acceso a datos (`JpaRepository` , consultas por nombre de método o `@Query`)
Service	`services`	Reglas de negocio y validaciones. Devuelven el mensaje de error, o `null` si salió bien
Controller	`controllers`	Reciben el pedido, llaman al service y eligen la vista o devuelven texto/JSON
View	`resources/templates`	HTML con Thymeleaf (`th:text` , `th:each` , `th:if`) + `static/css/umnatural.css`
Config	`config`	Interceptor de acceso, inicializadores y tareas programadas

---

5. Modelo de datos

Herencia de usuarios (JOINED)



- **JOINED** = una tabla por clase, unidas por el mismo `id` . Por eso **el correo de un particular está en `usuarios`** , no en `particulares` .
- Para verlos juntos se crearon **vistas** en Neon: `vista_particulares` , `vista_empresas` , `vista_turistas` .

Entidades del negocio

Entidad	Datos principales	Relaciones
`Emprendimiento`	nombre, ubicación, email, teléfono, descripción, **estado** (PENDIENTE / ACEPTADO / DENEGADO)	1:1 con Prestador, 1:n Actividades
`Actividad`	**título, descripción**, departamento, localidad, duración, precio, requisitos de edad, minutos previos para reservar, estado (ACTIVA / BAJA)	n:1 Emprendimiento, n:n TipoActividad, n:n TipoEquipamiento, 1:n Horario
`Horario` (débil)	fecha, hora de inicio y fin, cupos totales y disponibles, política de cancelación	Clave = actividad + fecha + hora de inicio
`Reserva` 🆕	cantidad de cupos, observación, estado (PENDIENTE_PAGO / CONFIRMADA / CANCELADA), fecha de creación, monto total	n:1 Turista, n:1 Horario (FK de **3 columnas**), 1:n Participante
`Participante` 🆕 (débil)	CI, nombre, apellido, fecha de nacimiento	Clave = reserva + CI
`TipoActividad`, `TipoEquipamiento`	nombre (único)	Datos globales que mantiene el admin

Todavía no existen: `Pago`, `Comentario`, `Departamento` (se usa una lista fija de los 19).

---

6. Roles y control de acceso

- **Login único:** `LoginController` busca el usuario por correo (sin distinguir mayúsculas), verifica la contraseña con BCrypt y guarda en la sesión `usuarioId`, `usuarioNombre`, `usuarioEmail` y `usuarioRol`. El rol se detecta con `instanceof`.
- **`ControlDeAccesoInterceptor`** (registrado en `WebConfig` para `/panel/**` y `/admin/**`) corta el pedido si el rol no corresponde:

Prefijo de la URL	Rol permitido
`/panel/prestador/**`	EMPRESA, PARTICULAR
`/panel/turista/**`	TURISTA
`/panel/admin/**`, `/admin/**`	ADMINISTRADOR

- Además, un prestador **solo puede tocar sus propias actividades** (`ActividadService.actividadDelPrestador`).
- Las contraseñas se guardan con **BCrypt**, nunca en texto plano.

---

7. Funcionalidades por rol

Público (sin login)

URL	Qué hace
`/`	Inicio: “Explorar actividades”, “Crear cuenta”, link para prestadores
`/actividades` 🆕	Buscador: palabra clave (sin distinguir mayúsculas ni tildes), departamento, tipo, fecha y precio máximo. Solo muestra actividades de emprendimientos **aceptados** y no dadas de baja
`/actividades/{id}` 🆕	Detalle: datos, categorías, equipamiento, próximas fechas con los lugares que quedan
`/registro/turista`, `/registro/prestador` → empresa / particular	Registros con validación
`/login`, `/logout`	Entrar y salir

Turista

URL	Qué hace
`/panel/turista`	Panel con accesos
`/panel/turista/reservar?...` 🆕	Elegir la cantidad de personas, cargar los datos de cada una (nombre, apellido, CI, fecha de nacimiento) y una observación. Total en vivo
`/panel/turista/reservas` 🆕	Mis reservas: estado, monto, participantes, hasta cuándo pagar

Prestador

URL	Qué hace
`/panel/prestador`	Estado del emprendimiento + accesos
`/panel/prestador/actividades` 🆕	Sus actividades: crear (solo si está **aceptado**), editar (si **no tiene reservas**), dar de baja, ver en el portal
`/panel/prestador/actividades/nueva` 🆕	Formulario: tipos y equipamiento con casillas **o escritos a mano** (se crean si no existen)
`/panel/prestador/actividades/{id}/horarios` 🆕	Agregar fechas con cupos y política de cancelación; eliminar si nadie reservó
`/panel/prestador/reservas` 🆕	Reservas recibidas, con el turista, los participantes y la observación

Administrador

URL	Qué hace
`/panel/admin`	Pendientes con **todos los datos** (tipo, razón social y RUT, o responsable y CI, email, teléfono, dirección, descripción) → Aceptar o Rechazar

URL	Qué hace
`/panel/admin/tipos` 🆕	Alta, modificación y baja de tipos de actividad y de equipamiento; no se borra uno en uso

---

8. Reglas de negocio implementadas

Regla	Dónde está
RNE #2 — reserva pendiente vence a las 2 h y libera el cupo	`ReservaService.cancelarReservasVencidas` (`@Scheduled`, cada 5 min)
RNE #3/#4 — cupos descontados de forma **atómica**, nunca negativos	`HorarioRepository.descontarCupos` (un solo `UPDATE ... WHERE cupos >= n`)
RNE #6 — horario con reservas no se elimina	`ActividadService.eliminarHorario`
RNE #8 — solo emprendimientos aceptados publican y aparecen	`ActividadService.puedePublicar`, `CatalogoService.estaPublicada`
RNE #9 — baja lógica	`ActividadService.darDeBaja` (estado BAJA)
RNE #10/#13 — RUT y CI únicos y válidos	`PrestadorService` / `PrestadorController`
RNE #11 — monto = precio × cupos	`ReservaService.crearReserva`
RNE #15 — actividad con reservas no se edita	`ActividadService.editarActividad`
Minutos previos para reservar (0 = hasta la hora de inicio)	`CatalogoService.sePuedeReservar`
Correo único sin distinguir mayúsculas	`UsuarioRepository.existsByUserEmailIgnoreCase`
Tipos globales: no se borran si están en uso	`TiposService`

El detalle técnico que más luce: si dos turistas reservan el último lugar al mismo tiempo, el `UPDATE` atómico deja pasar a uno solo. Al otro le aparece “Otro turista acaba de reservar esos cupos”. Se probó con dos pedidos simultáneos.

---

9. Tests unitarios

33 tests con JUnit 5 + Mockito. Los repositorios se reemplazan por objetos falsos, así no se necesita la base.

Clase	Tests	Qué cubre
`ReservaServiceTest`	9	cupos insuficientes, plazo vencido, faltan datos, CI repetida, no es turista, reserva correcta, otro turista se llevó los cupos, datos inválidos no descuentan, vencimiento a las 2 h

Clase	Tests	Qué cubre
`CatalogoServiceTest`	8	publicación, plazo para reservar (incluido 0 minutos), búsqueda sin tildes, filtros, próximos horarios
`ActividadServiceTest`	10	emprendimiento pendiente no publica, alta correcta, validaciones, edición bloqueada con reservas, actividad ajena, baja lógica, cupos del horario, horario pasado, eliminar horario con reservas, reservas recibidas
`TiposServiceTest`	6	no borrar en uso, borrar sin uso, nombres repetidos, alta, equipamiento en uso, tipos escritos a mano

En IntelliJ: clic derecho en `src/test/java` → **Run All Tests**.
`Proytic2026ApplicationTests.contextLoads` necesita conexión a Neon, así que puede fallar si no hay `DB_PASSWORD` configurado.

---

10. Base de datos en Neon: lo que hay que saber

- La URL está en `application.properties`; la contraseña llega por la variable de entorno `DB_PASSWORD`.
- `ddl-auto=update`: Hibernate **crea tablas y columnas nuevas solo**, pero **nunca borra columnas viejas**. Si alguien cambia una entidad y deja una columna obligatoria huérfana, los INSERT fallan (pasó con `actividades.permite_reembolso`). La solución es borrarla a mano.
- La base es **una sola para todo el grupo**: lo que crea uno lo ven todos.
- Consultas útiles en el SQL Editor:

```
        -- correo de un usuario
        SELECT id, nombre, apellido, correo FROM usuarios WHERE apellido ILIKE
'%peña%';
        -- columnas de una tabla (para encontrar columnas viejas)
        SELECT column_name, is_nullable FROM information_schema.columns WHERE
table_name = 'actividades';
```

---

11. Lo que falta

Código

Pendiente	Prioridad	Nota
Pago con pasarela externa (entidad `Pago`, servicio simulado que aprueba, rechaza o no responde, timeout, reintento)	**Obligatorio**	Hasta que exista, toda reserva queda PENDIENTE y se cancela a las 2 h
API REST pública (listar emprendimientos, actividades de un día por departamento, top 3 del mes, verificar emprendimiento, buscar por palabra clave) + **Swagger**	**Obligatorio**	
Comentarios y valoración 1–5	Alta	Los necesita el top 3
Verificación con admin, fecha y motivo del rechazo	Media	Relación *verifica* del MER
Cancelar horario o reserva con reembolso	Media	Depende del pago
Clima (Open-Meteo)	Opcional	
Limpieza: `UsuarioController` vacío, `EmprendimientoController` (`POST` público), páginas `alta*.html` rotas, sacar H2 del `pom.xml`	Baja	
Nombres en inglés (lo pide el enunciado)	Consultar al profe	Todo el código está en español

Entregables

Especificación de requerimientos con casos de uso (actualizar) · diagramas de secuencia · diagramas de flujo · diagrama de arquitectura · mockups (sirven capturas) · UML de clases · documentación de la API · manuales de usuario ×3.

---

12. Cómo trabajar con Git sin pisarse

```
        git fetch origin && git merge origin/main     # antes de empezar
        git add src .gitignore                         # no usar "git add ." (sube Claude outputs/, data/)
        git commit -m "qué hice"
        git push                                       # si pide pull: git pull y después git push
```

- Cada uno en su rama → Pull Request a `main` → otro lo revisa.
- `git push --force` solo en tu propia rama, nunca en `main`.

- Los archivos que tocan todos (`Actividad`, `umnatural.css`, el interceptor, `LoginController`) se avisan en el grupo antes de modificarlos.

---

13. Preguntas probables en la defensa

Pregunta	Respuesta
¿Cómo distinguen los roles si el login es uno solo?	Herencia JOINED + `instanceof` en `LoginController`; el rol queda en la sesión y el interceptor lo controla por prefijo de URL
¿Dónde está el correo de un particular?	En `usuarios`: con JOINED cada tabla guarda solo los campos de su clase
¿Cómo evitan sobrevender cupos?	`UPDATE` atómico con la condición `cupos >= n`; si devuelve 0 filas, no se crea la reserva. Todo dentro de `@Transactional`
¿Por qué Horario y Participante tienen clave compuesta?	Son entidades débiles del MER (`@IdClass`)
¿Por qué no se edita una actividad con reservas?	Para no cambiarle al turista el precio o las condiciones que aceptó (RNE #15)
¿Por qué la baja no borra?	Para conservar el historial de reservas (RNE #9)
¿Qué pasa con una reserva que no se paga?	A las 2 horas, una tarea `@Scheduled` la cancela y devuelve los cupos (RNE #2)
¿Cómo protegen las contraseñas?	BCrypt; en la base solo está el hash. La de Neon va por variable de entorno, no en el repo
¿Quién mantiene los tipos de actividad?	El admin (CU-A05). El prestador puede proponer uno nuevo escribiéndolo; queda global y el admin lo puede editar o borrar
¿Cómo testean sin base de datos?	Mockito reemplaza los repositorios; se prueban las reglas de los services

---

14. Archivos de referencia en el proyecto de Claude

- `que-falta-ultima-semana.md` — relevamiento completo de pendientes.
- `reparto-4-personas.md` — reparto del código.
- `explicacion-p2-turista.md` — explicación detallada de explorar y reservar.
- `explicacion-actividades-y-tipos.md` — explicación detallada de actividades y tipos.
- `mer-decisiones-y-rne.md`, `casos-de-uso.md` — modelo y casos de uso.